IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

One Buffer, Every Scope: A Fixed-Memory Scoped Symbol Table With Exact Undo and an Exact Refusal Law, Evaluated on Compiler-Generated Workloads

SHUBHI SINGH[1], ADHYAN JAIN[1], AKSHITH VENKATATARAMANA[1], and ARJUMAN[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India

Corresponding author: Shubhi Singh.

This work received no external funding.

ABSTRACT A compiler that runs inside a fixed memory budget needs a symbol table whose size is fixed in advance, whose behavior at the limit is defined, and whose cost per call cannot be driven up by hostile source text. Textbook and production designs rely on an allocator and offer none of these properties. We present STRATA, a scoped symbol table that keeps an open-addressing index, packed identifier records and an undo stack in one caller-supplied buffer; a call that does not fit is refused and leaves the table unchanged. We prove that, for every open-addressing scheme that never moves keys, removing the newest key by clearing its cell is an exact undo, so the table always equals a fresh table of its live names, and that last-in-first-out order is the only deletion order safe for every table. On this basis the probability that a probe limit refuses a legitimate name is computed exactly: at load three quarters it lowers the limit for a one-in-a-million refusal probability from 455 to 258 probes, and it agrees with the implementation in 2.8 million trials. On events logged by a C compiler for eight projects, STRATA gives no wrong answer, needs a fixed buffer 16.4 to 17.9 percent smaller than a conventional single-buffer table, a saving an exact space ledger attributes to a stored hash word and name packing, and is indistinguishable in speed on six of eight traces. Colliding identifiers slow declarations 300-fold; a keyed hash and the probe limit remove the slowdown.

INDEX TERMS Algorithmic complexity attacks, compilers, embedded software, hash tables, linear probing, memory management, probability, symbol tables.

I. INTRODUCTION

A SCOPED symbol table maps identifiers to declarations and forgets the declarations of a block when the block ends [1]. It is one of the oldest data structures in compiler construction [2], and for a compiler on a workstation it is a solved problem: a hash table from the standard library and an undo stack are enough [3]. The situation changes when the compiler itself is the constrained program. A compiler or interpreter front end that runs on a microcontroller, inside a boot loader, or in a tool that must never exceed a fixed allocation has three requirements that the standard solution does not meet.

Bounded memory. The table must live in storage whose size is fixed before the first declaration, without calls to an allocator whose fragmentation and failure behavior are outside the compiler's control.

Defined behavior at the bound. When a declaration does not fit, the table must say so and remain usable. A table that is left half-updated turns an ''out of memory'' diagnostic into an incorrect compilation.

Bounded work on untrusted input. If the source text is supplied by someone else, the cost of one table operation must not depend on that person's choice of identifiers. Hash tables are a known target of algorithmic complexity attacks [4], and seeding a common non-cryptographic hash does not prevent them [5].

The scoped tables we found in textbooks and in production compilers [3], [6], [7] obtain memory from an allocator, do not define their state after a failed allocation, and use unkeyed hashes. On the analytical side, the theory of open addressing covers insertions and deletions in arbitrary order [8]–[10]; the deletion order of a scoped table, last in first out, has not been treated as a case of its own, and probe limits are chosen with tail bounds whose looseness is unknown.

This article asks what it costs to meet all three requirements and how to configure a table that meets them, and answers by building, measuring and analyzing. The object of study is STRATA, a scoped symbol table in one buffer of 32-bit words. Its algorithm is the textbook one: one hash index, an undo stack, restoration on scope exit [3]. Our contributions lie in what is built around that algorithm, in what can be proved about it, and in how it is evaluated.

1) **A single-buffer scoped symbol table with an all-or-nothing refusal.** Index, interned names with their innermost declaration, hidden declarations and scope marks share one caller-supplied buffer. An operation that does not fit returns a status code and leaves every word of the state unchanged (Section IV).
2) **Exact undo and its converse.** For every open-addressing scheme that never moves a key, removing the newest key by clearing its cell restores exactly the previous table, so the table always equals the insertion-only table of its live names. For a given table the safe deletion orders are exactly the linear extensions of a dependency relation, and last-in-first-out order is the only one safe for every table (Section V).
3) **An exact refusal law for the probe limit.** With a static index and a limit of L probes per call, the probability that a legitimate name is refused is computed exactly at finite size, validated against the implementation, and turned into design values; the Chernoff-type bound used before is shown to be loose by a factor growing like $L^{3/2}$ (Section VI).
4) **A measured decomposition on compiler-generated workloads.** We separate what is gained by keeping everything in one allocation-free buffer, which a conventional chained table also achieves, from what the compact design adds, and an exact space ledger attributes the latter to a stored hash word and name packing rather than to interning. We also report that an apparent time penalty of the compact design was an implementation artifact (Section IX).
5) **An attack, its defenses, and a key-lifetime rule.** We construct C identifiers that collide in the default hash for every seed, measure the slowdown and the cost of a keyed hash and the probe limit, and show that the refusal law holds against chosen names only with a fresh key per table (Sections VI-C and IX-G).

We do not claim a new hashing scheme, a faster symbol table, or a result on hardware. The compact table is not faster than the conventional one; the refusal law is classical as a quantity and is credited as such; the collision construction applies a published technique [5]; and the microcontroller is emulated. Section XII lists what the evidence does not cover.

Section II reviews related work and Section III states the problem. Section IV describes the design, Section V the exact undo, and Section VI the bounded mode and its refusal law. Section VII describes how the guarantees were checked, Sections VIII and IX give the experimental setup and results, and Section X the interactive tool. Sections XI to XIII discuss, list limitations and conclude.

II. RELATED WORK

A. SCOPED SYMBOL TABLES

Graham *et al.* [2] described hashed symbol tables for languages with explicit scope control. The scheme used here is the one Appel presents for block-structured languages [3]: a single hash table maps each name to its innermost binding, an auxiliary stack records the names bound in each scope, and leaving a scope pops the stack and restores what was hidden. Lookup cost is independent of the nesting depth, in contrast to a stack of per-scope maps [1]. LLVM's `ScopedHashTable` [6] is a reusable implementation with nodes obtained from an allocator; lcc [11] links one table per scope level, and the Tiny C Compiler [7] keeps symbol stacks that are popped at scope exit. All of these allocate per symbol or per node, none defines the state of the table after an allocation fails, and none bounds the work of a call. Region allocation [12] removes most of the allocator's cost for compilers; a chained table whose entries live in a stack-like arena is its natural single-buffer form and is our main baseline. Microcontroller runtimes such as MicroPython intern identifiers in pools [13], which addresses the cost of names but not scopes or failure behavior.

B. DELETION IN OPEN ADDRESSING

Removing a key from a linear-probing table normally requires a tombstone or a backward shift [8], [14], and the way deletions are implemented strongly affects linear probing over time [9]. Sanders [15] gives the test for when a single deleted element must become a tombstone. Robin Hood hashing [16], used by the predecessor of this work [17], and cuckoo and hopscotch hashing [18], [19] move keys. That the most recently inserted key of linear probing can be removed by emptying its slot follows from the insertion rule and is likely folklore in backtracking code; what we add is the statement for every non-reordering scheme, the characterization of all safe orders, and their use for exact analysis. The property is related to, but weaker than, history independence [20], [21]: here the layout is a function of the live names and their order.

C. ANALYSIS OF OPEN ADDRESSING

For an ideal hash, Knuth analyzes insertion-only linear probing exactly [8]; the distribution of the unsuccessful-search cost, whose tail is the refusal probability studied here, is known through exact generating functions and Poisson limits [22]–[24], and the displacement law of Robin Hood hashing [25], [26] reappears in Section V as a law of orphaned keys. Recent work determines optimal probe complexity for open addressing without reordering [10] and the cost of dynamic workloads [27]. Alharbi, Banderier and Bouillaguet [28] analyze bounded linear probing, in which a key that exceeds the probe cap is discarded and redrawn; our event, refusal with no change, under last-in-first-out dele-

tions, is a different process. Pagh *et al.* [29] and Thorup [30] analyze linear probing under limited independence.

D. ALGORITHMIC COMPLEXITY ATTACKS

Crosby and Wallach [4] showed that an attacker who chooses the keys can degrade a hash table to a list. Aumasson and Bernstein [5] showed that MurmurHash2 and MurmurHash3 admit multicollisions that hold for every seed and proposed SipHash; the accompanying presentation [31] gives the construction for MurmurHash3. HalfSipHash with one compression and three finalization rounds is documented in the Linux kernel as suitable only for mitigating hash-table flooding [32]. Adaptive attacks through timing are an old observation [33], [34]. Compilers for trusted input commonly choose speed instead: the hash inside the Rust compiler is deliberately unkeyed [35]. We take the opposite case and measure both the attack and the defenses on a scoped table.

E. SUMMARY OF DIFFERENCES

Table 1 summarizes the properties relevant here. Entries for other systems state what their public documentation or source code shows; "no" means that we found no such mechanism.

III. PROBLEM FORMULATION

A. OPERATIONS AND SEMANTICS

A scoped symbol table supports four operations on a stack of scopes that initially holds one outermost scope.

- `enter`: push a new innermost scope.
- `exit`: remove the innermost scope and all declarations made in it; refused at the outermost scope.
- `declare`(x, v): bind name x to value v in the innermost scope. If x is already bound in that scope the value is replaced; otherwise a new declaration is created, which hides any declaration of x in an enclosing scope.
- `resolve`(x): return the value of the innermost visible declaration of x, or report that there is none.

A name is a nonempty byte string of at most 255 bytes. A value is a 32-bit integer with an 8-bit type tag. We write d for the number of live declarations, $n \leq d$ for the number of distinct names among them, s for the number of open scopes besides the outermost, and $w(x)$ for the number of 32-bit words that name x occupies.

B. REQUIREMENTS

R1 **Fixed storage.** All state resides in one buffer of C words supplied by the caller. No allocator is called.

R2 **All-or-nothing refusal.** Every operation either succeeds or returns a status code and leaves the complete state unchanged.

R3 **Correct scoping.** Every `resolve` returns what the definition above prescribes, given the operations that succeeded.

R4 **Bounded work per call.** A constant, independent of d and of the names chosen, bounds the number of index slots examined by one `declare` or `resolve`.

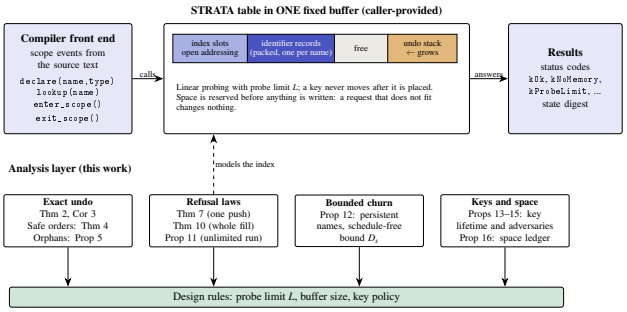


FIGURE 1. Architecture. The compiler front end issues four calls to a STRATA table that keeps its open-addressing index, packed identifier records and undo stack in one fixed buffer; the analysis of Sections V and VI turns the model of the index into design rules for the probe limit, the buffer size and the key policy.

R5 **Rare, quantified refusal of honest input.** Under R4, the probability that a declaration is refused although memory is available must be known and negligible for names that were not chosen to collide.

R1 to R3 are met by the default mode of STRATA and by the baseline. R4 and R5 are met by the bounded mode, R5 under the assumption stated next.

C. THREAT MODEL

The adversary supplies the source text and thereby every name and the order of all operations. The adversary knows the implementation, including the hash function, but not a 64-bit key that the device draws from a source the adversary cannot predict. The goal is to make table operations slow or, with a probe limit, to have legitimate declarations refused. We do not consider memory-safety attacks on the surrounding compiler or side channels through which the key might leak; Section VI-C states what an adversary who can observe probe lengths gains. Exhausting the buffer with many distinct names is always possible and is not an attack on the table: it is the budget doing its job, and R2 keeps the table usable afterwards.

IV. DESIGN OF STRATA

A. ARCHITECTURE AND LAYOUT

Fig. 1 shows the system and Fig. 2 the layout. All state lives in one array of 32-bit words. The *index* occupies the first m words, m a power of two. A slot is zero when empty; otherwise it holds an 8-bit fingerprint taken from the hash and a 24-bit reference to an identifier record. Collisions are resolved by linear probing, and the index is kept at most three quarters full.

The *identifier records* follow the index and grow upward, one per distinct visible name. A record holds the value of the name's *innermost* declaration, a word of metadata (number of name words, an encoding flag, the type tag, and the depth of the scope that made the innermost declaration), and the name. A lookup therefore ends in the record it finds, and its cost does not depend on how often the name is shadowed. Names consisting only of letters, digits and the underscore

TABLE 1. Properties of Scoped Symbol Tables and Identifier Stores Relevant to This Work.

Design	Scopes with shadowing	One caller-supplied buffer	Defined refusal, state unchanged	No rebuild inside a call	Bound on probes, exact refusal law	Keyed hash
Undo-stack scheme [3], LLVM `ScopedHashTable` [6]	yes	no	no	no	no	no
Tiny C Compiler symbol stacks [7]	yes	no	no	no	no	no
MicroPython interned strings [13]	no	pools	not stated	n/a	no	no
METIS-X [17] (predecessor)	yes	no	no	no	no	no
Conventional arena table (our baseline)	yes	yes	yes	no	no	no
STRATA, default mode	yes	yes	yes	no	no	optional
STRATA, bounded mode	yes	yes	yes	yes	yes	optional

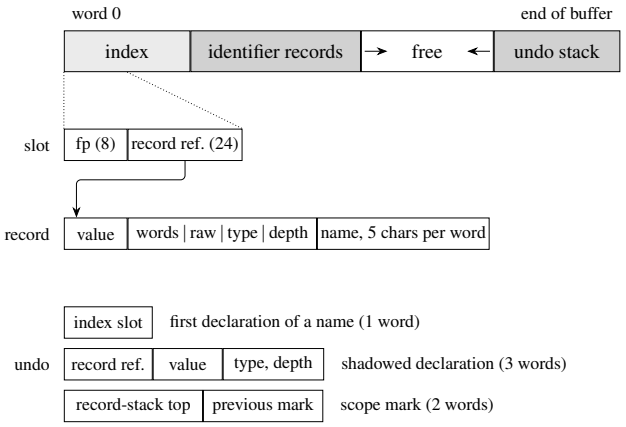


FIGURE 2. Layout of the buffer and of the three kinds of undo entries. The index and the identifier records grow from the start of the buffer, the undo stack from its end; the free space between them is shared.

are packed five characters to a word, six bits each; the query is packed once and compared word by word. Other names are stored as raw bytes.

The *undo stack* grows downward from the end of the buffer. A declaration of a name that was not visible pushes one word, the index slot of the new record. A declaration that hides an outer one pushes three words: the record reference and the value, type and depth that were overwritten. Entering a scope pushes a two-word mark holding the top of the record stack and the position of the previous mark. With m index slots the buffer is used up to

$$M = m + 1 + \sum_{x} \big(2 + w(x)\big) + n + 3\,(d - n) + 2\,s \quad (1)$$

words, where the sum runs over the n distinct visible names and the constant accounts for the reserved reference zero. The conventional table of Section VIII-B stores, for every declaration, a chain link, the hash, the value, a length-and-depth word and the unpacked name.

B. DECLARATION AND LOOKUP

Algorithm 1 gives the declaration. It has the form *search, reserve, write*: nothing is written before the last point at which the operation can be refused. A failed search ends at the first empty slot on the name's probe path, which is exactly where linear probing would insert the name; the slot is remembered and reused, so a declaration walks the path once.

Algorithm 1 Declaration of name x with value v

Input: name x, value and type v; current depth δ; probe limit L (bounded mode) or $L = \infty$

Output: a status; the state is unchanged unless the status is `ok` or `redeclared`

1: $k \leftarrow \mathrm{PackAndHash}(x)$ {refuses empty and over-long names}
2: $(r, e) \leftarrow \mathrm{Find}(k, L)$ {record, or first empty slot within L probes}
3: **if** $r \neq$ `nil` **and** $depth(r) = \delta$ **then**
4: overwrite value and type of r;
5: **return** `redeclared`
6: **else if** $r \neq$ `nil` **then**
7: **if** fewer than 3 free words **then**
8: **return** `no-memory`
9: **end if**
10: push $(r, value(r), type(r), depth(r))$ on the undo stack
11: overwrite value, type of r; $depth(r) \leftarrow \delta$;
12: **return** `ok`
13: **end if**
14: **if** fewer than $3 + w(x)$ free words, or the index would exceed its load limit and cannot grow **then**
15: **return** `no-memory`
16: **end if**
17: **if** $e =$ `nil` **then**
18: **return** `probe-limit`
19: **end if**
20: append a record for x with v and δ; store its reference and fingerprint in slot e
21: push e on the undo stack;
22: **return** `ok`

A lookup packs and hashes the name and probes from its home slot until it finds a slot whose fingerprint, metadata and name words match, or an empty slot. The fingerprint is a filter only; a match is always confirmed by comparing the name.

C. SCOPE EXIT WITHOUT TOMBSTONES

Leaving a scope pops the undo stack down to the scope's mark, newest entry first (Fig. 3). A three-word entry restores the value, type and depth it saved. A one-word entry empties the index slot it names. Then the top of the record stack is reset to the value in the mark, which discards the records created in

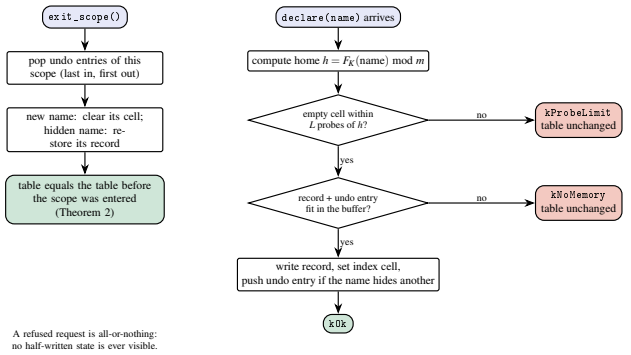


FIGURE 3. **Flow of a declaration and of a scope exit. A declaration is refused, with the table unchanged, when no empty slot is found within L probes or when the record and its undo entry do not fit; a scope exit pops the undo entries in reverse order.**

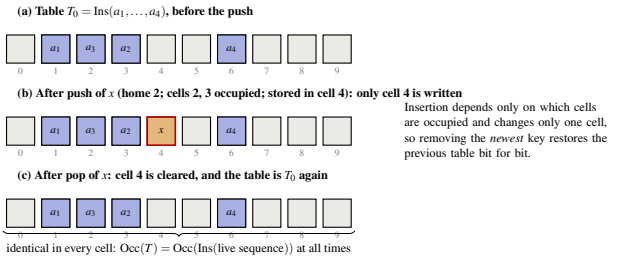


FIGURE 4. **Exact undo. Pushing x writes one empty cell; popping x clears it, and the table is identical in every cell to the table before the push.**

the scope. Emptying a slot is sufficient because of the order of removal: the key being removed is the newest key in the index, so it lies on no other key's probe path. Section V proves this for every non-reordering scheme and shows that no other order has the property. No tombstone is written and no key is shifted.

D. ALL-OR-NOTHING REFUSAL

Each operation validates its arguments, reserves everything it needs, and only then writes. In fixed mode the reservation is a comparison of counters; when the index must grow, the growth and the request are checked together before anything moves. After a successful reservation no step can fail. Consequently every refusal (no memory, nesting deeper than 16 383, an empty or over-long name, and the probe limit) leaves the state exactly as it was.

E. GROWING THE INDEX

In the default mode the index doubles when it would exceed its load limit. The identifier records are moved up as one block, which their relative references permit, and the index is rebuilt by reinserting the names oldest first, so that the property used in Section IV-C holds again. One-word undo entries name index slots, and those change. So that each record finds its undo entry again without a search and without scratch memory, which a fixed buffer does not have, each one-word entry and its record exchange one word before the rebuild and swap back during reinsertion. Every name is hashed once. When only a growable buffer is extended, the index and the records do not move and nothing is rehashed. In the default mode a single call that triggers a rebuild takes time proportional to n; the bounded mode removes this case.

V. EXACT UNDO AND SAFE DELETION ORDERS

Model the index as a ring of m cells. A rule is *non-reordering* if a key never moves after it is placed and the cell chosen for an insertion depends only on the set of occupied cells and on the key's own probe sequence; this covers linear, quadratic and double hashing and uniform probing, and excludes cuckoo, hopscotch and Robin Hood hashing. The operations are *push*, which inserts a name that is not live, and *pop*, which removes the most recently pushed live name by clearing its cell; a refused push changes nothing. In STRATA the pushes are the *first* declarations of names, since a shadowing declaration needs no index slot. For a sequence $a = (a_1, \ldots, a_t)$ of distinct names, $\mathrm{Ins}(a)$ is the table obtained by inserting them in this order into the empty table.

Theorem 1 (Exact undo): For every non-reordering rule, after any sequence of pushes and pops, with refused pushes leaving the state unchanged, the table equals $\mathrm{Ins}(a)$, where a is the live sequence in push order.

Proof: Induction on the number of operations. A refused push changes nothing. A successful push of x onto $\mathrm{Ins}(a)$ writes x into the cell that the rule chooses from the occupied set, which gives $\mathrm{Ins}(a{\cdot}x)$ by definition. For a pop, $a = u{\cdot}x$ and the table is $\mathrm{Ins}(u)$ with x additionally in the cell its insertion chose, because an insertion changes exactly one cell and that cell was empty in $\mathrm{Ins}(u)$; clearing it gives $\mathrm{Ins}(u)$ (Fig. 4). ■

The proof uses neither the lookup rule nor randomness, so it holds for adversarial names. For STRATA it gives a stronger, word-level statement that the test suite checks directly: after any sequence of operations, every word of the state equals the corresponding word of a fresh table into which exactly the still-live scope entries and declarations were made. Two consequences matter below. The table does not age: no sequence of scope entries and exits lengthens probe paths. And in the random-function model, for a schedule that does not depend on the hash, the law of the index at every moment is that of an insertion-only table with the current number of live names, so classical insertion-only results [8], [25] and the refusal law of Section VI hold at every moment.

Why is last-in-first-out order special? For a table built by $\mathrm{Ins}(x_1, \ldots, x_n)$ let $c(x)$ be the cell of x and say that y *depends on* x, written $y \lhd x$, if $c(x)$ lies strictly before $c(y)$ on the probe path of y (Fig. 5). A deletion order is *clear-safe* if, after every prefix of it has been cleared, every remaining key is still found.

Theorem 2 (Safe orders): Fix the homes of $x_1, \ldots, x_n$ ($2 \le n \le m$) under linear probing. (a) A deletion order is clear-safe for these homes if and only if it is a linear extension of $\lhd$, that is, y is deleted before x whenever $y \lhd x$. (b) It is clear-safe

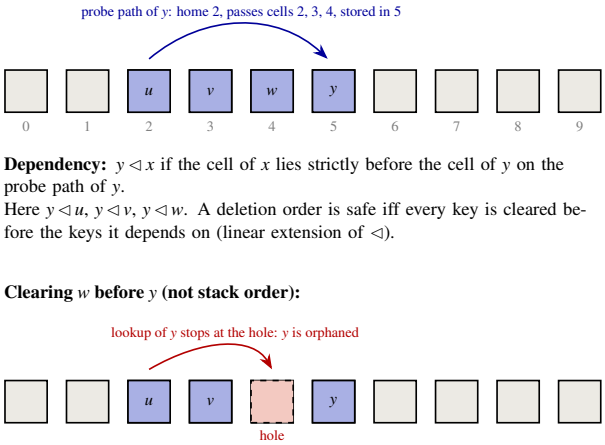


FIGURE 5. Dependency and orphans. Key *y* probed cells 2, 3 and 4 before it was stored in cell 5, so it depends on *u*, *v* and *w*. Clearing *w* first leaves a hole on its path and orphans *y*; clearing the newest key never does.

for every assignment of homes if and only if it is the reverse insertion order.

Proof: (a) Clearing never moves a key. The lookup of a remaining key y follows its probe path and stops at the first empty cell; the cells of that path before $c(y)$ were occupied, when y was inserted, by the keys x with $y \lhd x$. So y is found if and only if none of them has been cleared. (b) A key that depends on x is newer than x, so reverse insertion order is a linear extension of $\lhd$ for every assignment. Conversely, for $i < j$ give $x_1, \ldots, x_{j-1}$ distinct homes and give x_j the home of x_i; then $x_j \lhd x_i$, so a universally safe order deletes x_j before x_i for every pair $i < j$. ■

Part (a) holds verbatim for every greedy probing scheme. The number of keys that depend on a key quantifies the cost of leaving the discipline: it equals $Y_c - 1$, where Y_c follows the queue recursion $Y_c = (Y_{c-1} - 1)^+ + A_c$ with A_c the number of keys homed at cell c, and for $n/m \to \alpha$ its law converges to that of a discrete M/D/1 queue, with mean and fraction of independent keys

$$\mathbb{E}[\mathrm{orph}] = \frac{\alpha}{2(1-\alpha)}, \qquad \mathbb{P}(\mathrm{orph} = 0) \to \frac{(1-\alpha)(e^{\alpha}-1)}{\alpha}. \tag{2}$$

At load $3/4$ only 37 percent of keys could be removed out of order without harm, and clearing a random key orphans 1.5 keys on average (Table 2). The limit law is Janson's Robin Hood displacement law [25]; the reading as orphans of a deletion and the finite-size form are new as stated. This is why every deletion method for arbitrary order needs tombstones or shifting [8], [9], and why STRATA could not adopt a relocating deletion: its undo entries store index slots.

VI. BOUNDED MODE AND THE EXACT REFUSAL LAW

TABLE 2. Limit Law of the Number of Keys That Depend on a Uniformly Random Live Key, $\mathbb{P}(\mathrm{orph} = j)$, and Its Mean $\alpha/(2(1-\alpha))$.

α	j=0	1	2	3	4	mean
0.25	0.852	0.131	0.015	0.002	0.000	0.167
0.5	0.649	0.245	0.076	0.022	0.006	0.500
0.75	0.372	0.259	0.156	0.090	0.052	1.500
0.9	0.162	0.153	0.128	0.104	0.085	4.500

A. STATIC INDEX AND PROBE LIMIT

In the bounded mode the caller fixes the number of index slots m when the table is created. The index is never rebuilt; a declaration that would exceed the load limit is refused like one that exceeds the buffer. The caller may also set a probe limit L. A new name is inserted only if an empty slot lies among the first L slots of its probe path, and is refused with `probe-limit` otherwise (Algorithm 1). Since a stored name never moves and scope exit only empties slots, every stored name lies within L slots of its home, and a search may stop after L slots. One call thus examines at most L index words and, per fingerprint match, compares at most 64 name words (requirement R4). A name that hides a visible one needs no slot and is never refused for this reason. The bound does not cover scope exit, which is proportional to the declarations in the scope, nor name packing, which is linear in the name length.

B. HOW OFTEN DOES THE LIMIT REFUSE AN HONEST NAME?

By Theorem 1, the refusal probability at a moment with n live names is that of an insertion-only linear-probing table with n keys: the probability that a new key with an independent uniform home finds its first L cells occupied. For $g \geq 1$ let $B(g) = g^{g-1}/g!$ and, for $j \geq 1$, $Q_j(s) = \frac{j}{s}\, s^{s-j}/(s-j)!$.

Theorem 3 (Exact refusal law): Let a ring of m cells hold $n = m - k$ keys, $k \geq 1$, with independent uniform homes under linear probing. The probability that a new key with an independent uniform home finds its first L cells occupied is

$$R_m(n, L) = \frac{k}{m} \sum_{g=L+1}^{n+1} (g - L)\, \pi_g, \qquad \pi_g = \frac{B(g)\, Q_{k-1}(m-g)}{Q_k(m)}. \tag{3}$$

Proof sketch: Cut the ring at its k empty cells. Each empty cell and the run of occupied cells after it form a block; by the cycle lemma [36] block sizes are Borel distributed, and, conditioned on filling the ring, the k blocks are exchangeable, so the first has size g with probability π_g by the Borel–Tanner identity [37]. A new key is refused exactly when its home starts a window of L occupied cells; a block of g cells contains $(g-L)^+$ such homes, and the expected number of blocks is k. The full proof is in the accompanying theory manuscript [38]. ■

$R_m(n, L)$ is nondecreasing in n, so the value at the load limit covers every lower load. As $n/m \to \alpha$ it converges to $(1 -$

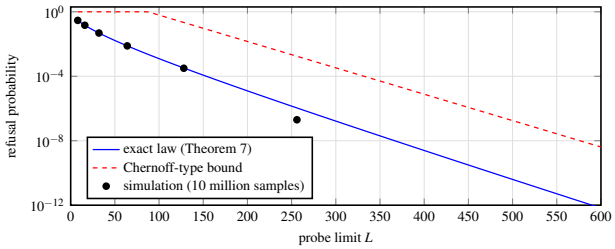


FIGURE 6. Probability that an insertion needs more than L probes ($m = 65\,536$, load $3/4$): exact law (Theorem 3), the Chernoff-type bound, and a stored simulation of 10 million samples from 500 tables.

TABLE 3. Smallest Probe Limit L With Per-Insertion Refusal Probability at Most ε: Exact Law for Index Sizes m, Its Limit, and the Limit the Chernoff-Type Bound Would Require.

load	ε	m=2^8	2^{10}	2^{12}	2^{14}	2^{16}	$m \to \infty$	Chernoff
3/8	10^{-6}	25	28	29	29	29	29	43
3/8	10^{-9}	37	44	46	47	47	47	62
1/2	10^{-6}	42	49	51	52	52	52	81
1/2	10^{-9}	59	77	83	84	85	85	117
5/8	10^{-6}	70	93	101	104	104	104	171
5/8	10^{-9}	95	142	162	168	170	170	244
3/4	10^{-6}	121	201	242	255	258	260	455
3/4	10^{-9}	148	289	380	413	422	426	638

$\alpha) \sum_{g>L} (g-L)\,\mathrm{Borel}(g;\alpha)$, and with $\rho = \alpha e^{1-\alpha}$

$$R_\infty(\alpha, L) \sim \frac{(1-\alpha)\,\rho^{L+1}}{\alpha\sqrt{2\pi}\,(1-\rho)^2\,L^{3/2}} \qquad (L \to \infty). \tag{4}$$

The quantity is classical: it is the tail of the unsuccessful-search cost, whose generating function and Poisson limit are given by Janson and Viola [24] and whose finite-size values follow from Knuth's run-length counting [8]. What is new here is the short derivation of the explicit sum (3), its validation against the implementation, and its use as a design rule. A Chernoff bound on the number of homes in an interval, which an earlier version of this study used, gives $\rho^L/(1-\rho)$; by (4) it is loose by a factor growing like $L^{3/2}$. At load $3/4$ and 65 536 slots the bound exceeds the exact value by a factor of 315 at $L = 64$ and 4 637 at $L = 512$.

Table 3 turns the law into design values by bisection on L. At load $3/4$ and $\varepsilon = 10^{-6}$ the exact limit is 258 for 2^{16} slots and 260 in the limit, against 455 for the Chernoff-type rule, 43 percent fewer probes allowed per call; it is smaller for small indexes (121 at 256 slots). For a compilation that declares N fresh names at most $N\,R_m(n_{\max}, L)$ of them are refused in expectation. The theory manuscript also gives the exact probability that a whole fill is refusal-free, which lowers the whole-fill limit by about a quarter against the union bound, the probability that a persistent set of names can never refuse a fresh one, and a schedule-free bound for bounded churn above persistent names [38].

TABLE 4. Honest Names Refused by the Probe Limit (Percent of 700) After the Adversary's Learned Colliders Are Declared: Persistent Key (Mean and Maximum Over Five Seeds), Fresh Key, and Random Names.

colliders	persistent key (mean)	(max)	fresh key	random names
400	1.00%	1.29%	0.00%	0.00%
512	5.51%	6.14%	0.00%	0.00%
600	4.71%	6.00%	0.00%	0.00%
700	5.14%	5.71%	0.00%	0.00%
800	5.14%	6.00%	0.00%	0.00%

C. KEYED HASH AND THE LIFETIME OF A KEY

The default hash is the 32-bit MurmurHash3 mixer [39] applied to the name words. As an option, the table hashes with HalfSipHash with one compression and three finalization rounds [5], [32] under a 64-bit key supplied by the caller, checked against the reference code [40]. We rely on the function's design goal, indistinguishability from a random function for an adversary without the key, and make no cryptographic claim.

Theorem 3 applies to chosen names only if the key is independent of everything the adversary has seen. If every table is created with a fresh key and the adversary fixes its names before the key is drawn, the homes of all names are independent and uniform, and the law holds verbatim for every name set and scope schedule. If one key is reused across tables and the adversary can observe probe lengths, it can learn names that collide with a target cell and plant a long run of occupied cells: in an experiment against the real table with an idealized probe-length oracle ($m = 2\,048$, $L = 512$, 700 honest names, five seeds), 1 to 5.5 percent of honest names were refused with a persistent key and none with a fresh key or with the same number of random names (Table 4). The rule is therefore one key per translation unit, derived from a device secret and a never-repeating counter. The work bound of R4 does not depend on the key: a call examines at most L slots whatever the names.

D. COLLIDING IDENTIFIERS FOR THE DEFAULT HASH

The mixer processes a name word k as $h \leftarrow \mathrm{rotl}_{13}\big(h \oplus f(k)\big) \cdot 5 + c$, where f multiplies, rotates and multiplies and is a bijection on 32-bit words that does not involve the seed. Following Aumasson and Bernstein [5], [31], flipping bit 18 of $f(k_1)$ flips bit 31 of the rotated word; multiplying by 5 and adding c leaves that difference in bit 31 for every h; and flipping bit 31 of $f(k_2)$ in the next word cancels it. Two consecutive words can thus be exchanged without changing the hash, whatever the seed. Here a word must be a packing of five identifier characters, and a random search finds word pairs that satisfy the constraints for both alternatives within a fraction of a second. Concatenating t such pairs gives 2^t distinct identifiers of $10t$ characters with one common hash; with $t = 15$ this yields 32 768 valid C identifiers of 150 characters, which collide under the built-in seed and eight further seeds.

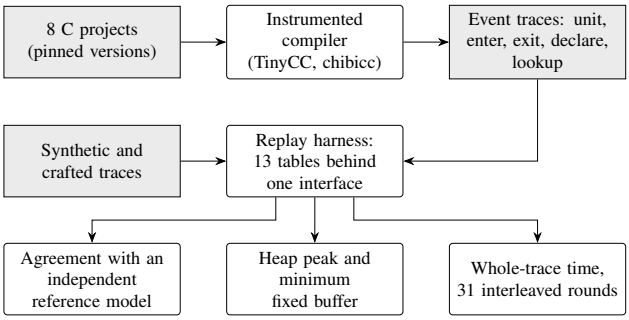


FIGURE 7. Evaluation method. Symbol-table events are logged by instrumented compilers and replayed on every table; an independent reference model decides correctness.

VII. VERIFICATION

Unit and randomized differential tests. A test program compares the table with an independent reference model after every operation, in configurations that cover packed and raw names, fixed and growable buffers, a dense and a sparse index, failing allocators, and a hook that forces every key into one probe cluster. It requires a 64-bit digest of the complete state to be unchanged after every refusal and checks the word-level form of Theorem 1 by rebuilding a fresh table from the live declarations and comparing memory. It makes 2 732 358 checks under AddressSanitizer and UndefinedBehaviorSanitizer with two compilers.

Bounded exhaustive enumeration. A second program enumerates every sequence of nine operations over {enter, exit, declare a, b, c, d} in eight small configurations chosen so that every kind of refusal occurs: 80 621 568 sequences, 725 594 112 operations, 130 361 632 of them refused, no failure.

Differential fuzzing and mutation. A libFuzzer harness ran 9.8 million executions on the code before the change of Section IX-E and 2.5 million on the final code without a failure, and detected each of 9 injected single-line defects within a second.

The analysis. Theorem 1 was checked in 488 919 state comparisons over five insertion rules, Theorem 2 on every assignment and every deletion order of small rings (397 562 pairs), and the refusal law against brute-force enumeration of every home sequence in exact rational arithmetic (258 cases) and against an independent derivation by Knuth's Abel-binomial route (481 cases). The proofs are pen-and-paper; no proof assistant was used.

VIII. EXPERIMENTAL SETUP

A. WORKLOADS

The workloads are sequences of symbol-table events produced by a real compiler (Fig. 7). We added logging to the Tiny C Compiler [7] where it looks up an identifier, pushes a symbol, and opens or closes a scope, and compiled eight open-source C projects at pinned versions (Table 5). At the start of each translation unit every table is destroyed and created anew, as a compiler would do. A second set of seven traces from an instrumented chibicc [41] is used for the bounded-mode experiment, and nine synthetic traces stress cases that C programs rarely produce.

TABLE 5. The Eight Compiler Traces. "Names" Counts Distinct Identifiers; "Peak" Is the Largest Number of Simultaneously Live Declarations.

Project	Units	Events	Declares	Lookups	Names	Peak	Depth
cJSON	2	20 874	5 247	9 194	1 366	1 067	10
littlefs	2	21 080	3 835	12 230	1 001	732	16
zlib	15	58 058	16 341	27 405	1 041	571	23
Lua	34	188 509	58 619	86 887	3 622	1 365	14
FreeRTOS	9	37 875	10 346	17 417	2 226	1 505	16
Mbed TLS	105	983 008	319 886	438 876	7 622	2 302	14
TinyCC	1	89 135	13 776	54 623	6 704	5 509	27
SQLite	1	268 960	27 072	172 426	7 801	4 245	22

B. TABLES COMPARED

13 tables are driven through one interface.

- STRATA and four variants: raw instead of packed names, a sparse index (at most half full), no shrinking of the buffer, and the keyed hash.
- *The conventional arena table*, our main baseline: bucket heads, a stack of entries (chain link, hash, value, length and depth, unpacked name) and scope marks in one buffer, with the same storage discipline as STRATA. It was written as the strongest conventional design we could build, so that a comparison isolates the compact layout from the mere absence of an allocator.
- One standard-library hash map per scope; an undo log over a standard-library map (the algorithm of LLVM's `ScopedHashTable`); the same over a Swiss-table map [42]; an interning Swiss-table map.
- METIS-X [17], the predecessor, a Robin Hood table with one slot per declaration, and two further tables from its repository, used unmodified.

C. METRICS AND PROCEDURE

Correctness is agreement with the reference model on every lookup and declaration result. *Heap peak* counts every allocation with its usable size. The *smallest fixed buffer* is found by bisection, at a resolution of 64 bytes, as the smallest buffer with which the whole trace replays without a refusal. *Replay time* is the time to replay a whole trace on a fresh table. In each of 31 measured rounds, after two warm-up rounds, every table replays the trace once and the starting table rotates. For two tables we report the median over rounds of the ratio of their times with a 95 percent bootstrap interval from 2000 resamples, and call a difference *established* when the interval excludes zero ("n.e." otherwise).

D. ENVIRONMENT

All host measurements were taken on one machine: Intel Core Ultra 7 258V, Windows Subsystem for Linux 2 with Ubuntu 22.04 (Linux 6.18.33.2-microsoft-standard-WSL2), g++ 11.4.0 with `-O2`, the benchmark pinned to one core, run `20261013-143828-paper` of the public artifact [38].

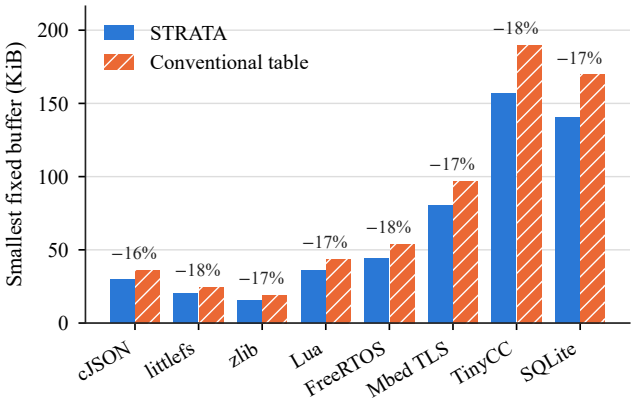


FIGURE 8. Smallest fixed buffer with which each compiler trace replays without a refusal. The label gives the saving of STRATA over the conventional arena table.

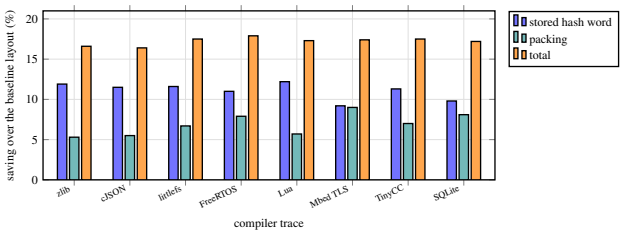


FIGURE 9. Saving of STRATA's layout over the conventional single-buffer layout on the eight traces, split into the stored hash word and name packing, from the exact space ledger.

The embedded experiment uses `arm-none-eabi-g++` 10.3.1 with `-Os` and QEMU [43] 6.2 emulating a Cortex-M3 board (`mps2-an385`) restricted to 64 KiB of memory, without heap and without operating system. The refusal law was checked against the real bounded-mode table, with the keyed hash and a fresh random key per trial, in seven settings of 400 000 trials.

IX. RESULTS

A. CORRECTNESS

All 13 tables agree with the reference model on the eight Tiny C Compiler traces and on the seven chibicc traces: 195 table and trace pairs, no wrong lookup, no wrong declaration result, no refused declaration. On the synthetic traces METIS-X refuses 149 of 400 declarations when one name is shadowed in 400 nested scopes.

B. MEMORY

STRATA replays the traces in fixed buffers of 15.8 to 156.5 KiB, which is 27 to 36 bytes per live declaration at the peak (Fig. 8, Table 6). The conventional arena table needs a buffer that is larger on every trace, so that STRATA saves 16.4 to 17.9 percent; Section IX-C explains where this saving comes from.

Against tables that allocate, the difference is larger and has a different cause. The heap peak of STRATA in growable mode is 23 to 76 percent (median 58 percent) below that of METIS-X, and STRATA makes 98.7 to 99.9 percent fewer allocator calls. In growable mode the conventional arena table reaches the same heap peak as STRATA on 6 of the 8 traces, because both grow one block in steps. Most of the gain over the predecessor is therefore due to keeping everything in one allocation-free buffer, not to the compact layout.

C. ANATOMY OF THE SAVING: AN EXACT SPACE LEDGER

Every single-buffer layout considered here uses a number of words that is a sum of per-element costs, as in (1). Let C_0 be the conventional layout, C_1 the same without the stored hash word, S_{raw} STRATA's layout with raw names and S STRATA itself. Subtracting the formulas gives exact identities at every instant:

$$U_{C_0} - U_{C_1} = d, \qquad U_{S_{\mathrm{raw}}} - U_S = \sum_{x \text{ packable}} \left(\left\lceil \frac{\ell_x}{4} \right\rceil - \left\lceil \frac{\ell_x}{5} \right\rceil \right), \tag{5}$$

and interning with an undo log saves the names of hidden declarations but costs one extra word per open scope. The accounting reproduces all 24 measured minimum buffers to the 64-byte resolution of the measurement, so it can be trusted for the hypothetical layout C_1. On the eight traces, dropping the stored hash saves 9.2 to 12.2 percent and packing a further 5.3 to 9.0 percent, while interning changes the footprint by between -0.38 and 0.00 percent (Fig. 9, Table 7). Against a conventional table that does not store the hash, STRATA would be only 5 to 9 percent smaller, all of it from packing; interning pays only in programs that shadow often, as the sweep of Section IX-I confirms.

D. REPLAY TIME

Table 8 gives the paired comparisons. Against the conventional arena table, STRATA is faster on 0 traces, slower on 2, and on 6 the difference is not established; the point estimates lie between -10.6 and $+10.7$ percent. The compact table is not faster than the conventional one, and where a difference is established it is at most 10.7 percent. Against METIS-X, STRATA is faster on 8 of 8 traces, by 15.9 to 36.7 percent, and the conventional arena table is faster than METIS-X on 8 of 8, by 14.6 to 34.8 percent. Per operation (Table 9), the 95th-percentile latency of a declaration is 56 to 66 percent lower with STRATA than with METIS-X and that of a scope exit 71 to 82 percent lower; lookup latency differs by between -20.3 and $+4.9$ percent without a consistent direction.

E. A TIME PENALTY THAT WAS AN ARTIFACT

An earlier version of STRATA was slower than the conventional arena table on every trace, by 11.4 to 33.8 percent (Fig. 10), and we initially reported this as the price of the compact design. Per-operation latencies contradicted that reading: lookups were level and the whole difference was in declarations. The cause was rebuild work of three kinds: the index was rebuilt on every growth of the buffer, although only the undo stack moves in that case; each rebuild converted the

TABLE 6. Memory on the Eight Compiler Traces. Smallest Fixed Buffer of the Two Single-Buffer Tables, and Heap Peak in Growable Mode of Six Tables (KiB).

	Smallest fixed buffer				Heap peak, growable mode (KiB)					
Project	STRATA (KiB)	Arena (KiB)	Saving (%)	Bytes per live decl.	STRATA	Arena	METIS-X	Map per scope	Undo log	Interning map
cJSON	30.0	35.9	16.4	28.8	42.9	42.8	102.7	72.0	116.0	140.4
littlefs	20.0	24.2	17.5	28.0	25.8	25.7	106.9	54.1	67.8	141.9
zlib	15.8	18.9	16.6	28.2	17.2	25.7	52.6	44.4	60.3	71.1
Lua	35.9	43.4	17.3	27.0	64.2	96.2	106.8	96.7	127.7	142.3
FreeRTOS	44.0	53.6	17.9	29.9	57.8	57.7	217.3	116.1	147.3	293.1
Mbed TLS	80.0	96.9	17.4	35.6	144.3	144.2	234.4	189.4	277.7	310.9
TinyCC	156.5	189.8	17.5	29.1	324.5	324.4	423.0	424.5	550.8	575.6
SQLite	140.4	169.4	17.2	33.9	194.8	194.7	451.0	333.3	513.5	1125.6

TABLE 7. Smallest Fixed Buffer (KiB) of Five Layouts and the Savings (Percent) of Dropping the Stored Hash ($C_0 \to C_1$), of Packing ($C_1 \to S$) and in Total ($C_0 \to S$).

trace	C_0	C_1	C_2	S_{raw}	S	hash	pack	total
zlib	18.9	16.6	16.6	16.7	15.8	11.9	5.3	16.6
cJSON	35.9	31.8	31.8	31.8	30.0	11.5	5.5	16.4
littlefs	24.2	21.4	21.4	21.5	20.0	11.6	6.7	17.5
FreeRTOS	53.6	47.8	47.8	47.8	44.0	11.0	7.9	17.9
Lua	43.4	38.1	38.1	38.1	35.9	12.2	5.7	17.3
Mbed TLS	96.9	87.9	87.9	87.9	80.0	9.2	9.0	17.4
TinyCC	189.8	168.2	168.2	168.3	156.5	11.3	7.0	17.5
SQLite	169.4	152.8	152.8	152.9	140.4	9.8	8.1	17.2

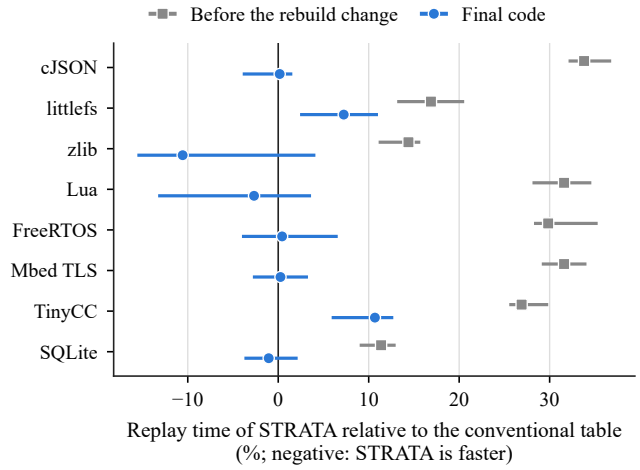


FIGURE 10. Replay time of STRATA relative to the conventional arena table before and after the rebuild change of Section IX-E: median of paired ratios with 95 percent bootstrap interval.

one-word undo entries from slots to record references and back, hashing every name twice; and a new name walked its probe path twice. After the changes described in Section IV-E the penalty is gone on most traces and small on the rest. The changes alter no result other than time. We report the episode because the first measurement would have supported a plausible but wrong conclusion, and because a baseline strong enough to expose it prompted the diagnosis.

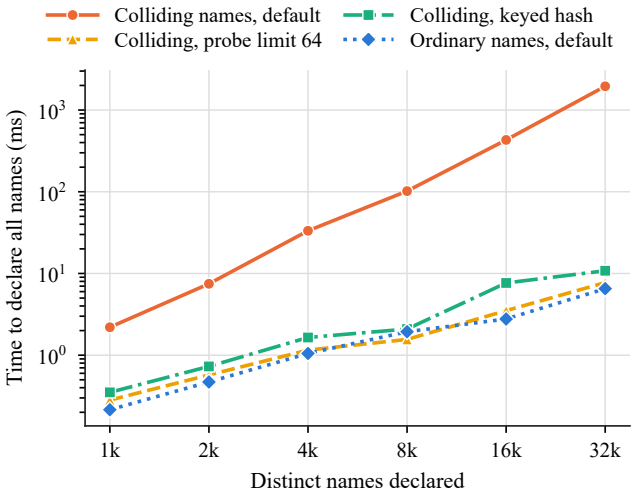


FIGURE 11. Time to declare *N* distinct names of 150 characters. Colliding names are those of Section VI-D; ordinary names are random identifiers of the same length. Both axes are logarithmic.

F. ABLATION AND SYNTHETIC TRACES

Table 10 isolates three design choices. Storing names as raw bytes instead of packing them needs a 5.5 to 9.0 percent larger buffer and is slower on 2 of 8 traces and faster on 0, because a packed name has fewer words to hash and compare. Keeping the index at most half full makes replay faster on 6 of 8 traces (changes between −4.5 and −12.9 percent) and needs a 18 to 27 percent larger buffer, which more than cancels the memory advantage: index load is the lever that trades the saving for speed. On the 9 synthetic traces (Table 11) STRATA is faster than the conventional table on 6 and slower on 1: faster where names are hidden often, where scopes are entered and left at a high rate, and with short names, and slower on the trace dominated by lookups.

G. HASH FLOODING AND ITS DEFENSES

We declare and then look up the 32 768 colliding identifiers and as many random identifiers of the same length (Fig. 11, Table 12). In the default configuration the colliding names take 1954 ms to declare against 6.5 ms for ordinary names, a factor of 300, and 1684 ms to look up; time grows with exponent 1.96 in the number of names, since every new name scans the cluster formed by all earlier ones. With the keyed

TABLE 8. Replay Time on the Eight Compiler Traces: Median Ratio Over 31 Paired Rounds, in Percent, With 95 Percent Bootstrap Interval. Negative Means That the First Table Is Faster; "n.e." Means Not Established.

Project	STRATA vs. arena table		STRATA vs. METIS-X		Arena table vs. METIS-X		Keyed vs. default hash
cJSON	+0.2 [−3.9, +1.6]	n.e.	−22.8 [−27.5, −19.1]	faster	−21.3 [−23.8, −19.8]	faster	+20.2 [+18.5, +24.6]
littlefs	+7.2 [+2.4, +11.0]	slower	−24.5 [−27.2, −18.1]	faster	−26.5 [−28.9, −22.7]	faster	+24.6 [+18.5, +29.8]
zlib	−10.6 [−15.6, +4.1]	n.e.	−29.9 [−32.8, −27.4]	faster	−20.4 [−26.9, −15.6]	faster	+32.2 [+16.1, +41.4]
Lua	−2.7 [−13.3, +3.6]	n.e.	−30.9 [−35.3, −29.2]	faster	−28.4 [−33.8, −25.2]	faster	+22.7 [+19.1, +35.2]
FreeRTOS	+0.4 [−4.0, +6.6]	n.e.	−33.9 [−39.0, −31.3]	faster	−34.8 [−36.5, −32.9]	faster	+20.3 [+15.2, +22.4]
Mbed TLS	+0.2 [−2.8, +3.3]	n.e.	−36.7 [−39.6, −32.4]	faster	−34.8 [−38.3, −32.2]	faster	+27.1 [+23.0, +34.3]
TinyCC	+10.7 [+5.9, +12.7]	slower	−20.4 [−25.7, −18.7]	faster	−27.5 [−33.7, −22.3]	faster	+20.6 [+13.1, +22.0]
SQLite	−1.1 [−3.7, +2.1]	n.e.	−15.9 [−20.7, −6.3]	faster	−14.6 [−19.5, −10.0]	faster	+22.3 [+18.7, +33.8]

TABLE 9. 95th-Percentile Latency per Operation in Nanoseconds for STRATA (S), the Conventional Arena Table (A) and METIS-X (M).

	Lookup			Declare			Scope exit		
Project	S	A	M	S	A	M	S	A	M
cJSON	49	41	49	62	71	141	13	19	56
littlefs	51	41	49	65	72	187	19	26	90
zlib	44	41	44	56	68	136	16	23	59
Lua	53	89	54	66	130	164	21	56	84
FreeRTOS	52	44	54	68	73	180	15	23	87
Mbed TLS	66	54	69	79	78	202	27	36	109
TinyCC	51	44	51	66	75	188	20	24	76
SQLite	52	53	65	71	86	204	22	28	76

TABLE 10. Ablation: Change Relative to Default STRATA in Smallest Fixed Buffer and in Replay Time (Percent; Verdict From the 95 Percent Interval).

	Raw names		Sparse index		No shrinking
Project	Buffer	Time	Buffer	Time	Time
cJSON	+5.8	−0.2 (n.e.)	+26.7	−12.1 (faster)	−2.8 (faster)
littlefs	+7.5	+4.4 (n.e.)	+20.0	−12.9 (faster)	+0.7 (n.e.)
zlib	+6.0	+3.1 (n.e.)	+25.4	−9.0 (faster)	−4.9 (faster)
Lua	+6.1	+7.1 (slower)	+22.3	−11.8 (faster)	+1.4 (n.e.)
FreeRTOS	+8.5	+3.0 (n.e.)	+18.2	−9.3 (faster)	−4.4 (n.e.)
Mbed TLS	+9.9	−1.6 (n.e.)	+20.0	−4.8 (n.e.)	+2.7 (slower)
TinyCC	+7.5	+9.9 (slower)	+20.4	−9.2 (faster)	+1.2 (n.e.)
SQLite	+8.9	−0.2 (n.e.)	+22.8	−4.5 (n.e.)	+2.9 (n.e.)

hash the factor is 1.2: names built against the unkeyed mixer are ordinary names under a key their author does not know. With the unkeyed hash and a probe limit of 64, time grows linearly, at the price that 32 704 of the 32 768 colliding names are refused. The limit bounds the work of a call whatever the hash does, and a fresh key keeps an adversary from causing refusals (Section VI-C). On the compiler traces the keyed hash increases replay time by 20.2 to 32.2 percent (Table 8).

H. THE REFUSAL LAW AGAINST THE IMPLEMENTATION AND THE TRACES

Table 13 and Fig. 12 compare the exact law with the observed frequency of a refused push in the real table, with the keyed hash and a fresh key per trial. All seven z-scores lie within ± 1.96 (largest $|z| = 1.96$), and a table with the probe limit and one without, run in lock step, never disagreed in the clean trials. The law therefore describes the implementation, not only the model. A stored simulation of 240 000 000 insertions also agrees with it (Fig. 6).

The compiler traces alone would have suggested a much smaller limit. Replaying the 15 compiler traces (719 833 declarations) in bounded mode, with the index sized for each trace's peak, a limit of 64 refuses no declaration with either hash and the longest probe sequence is 47 (Table 14). Yet at the load limit of $3/4$, 0.77 percent of fresh names need more than 64 probes: the traces' tables are small and spend most of their time well below the load limit. The exact law gives the setting directly: 258 for a refusal probability of 10^{-6} per declaration at 2^{16} slots, or 52 with an index kept at most half full. The limit of 512 recommended in the library documentation is safe but conservative: at 65 536 slots it corresponds to a refusal probability of 2.4×10^{-11} per declaration.

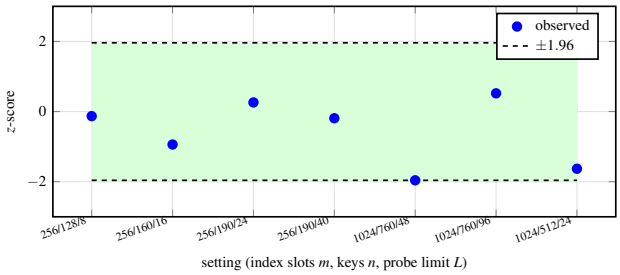


FIGURE 12. Agreement of the exact refusal law with the real bounded-mode table: standard scores in seven settings of 400 000 trials each, all within plus or minus 1.96.

I. WHEN DOES THE SMALLER BUFFER MATTER?

A saving of one sixth decides whether a workload fits only when the budget lies between the needs of the two tables. Of the 169 translation units of the eight projects, the units that only STRATA can hold are few at every budget, at most 23 at 16 KiB (Table 15). On the emulated microcontroller (Section IX-K), three whole workloads fit a buffer in which the conventional table refuses operations. A sweep over the shadowing factor (live declarations per distinct name) and the name length confirms (1) and the ledger: the saving grows with shadowing, since a hidden declaration costs three words in STRATA and a full entry with the name in the conventional table, and reaches 65 percent at the highest factor tested (Table 16). Replay time relative to the conventional table follows the name length rather than the shadowing factor.

TABLE 11. Synthetic Stress Traces: Replay Time of STRATA Relative to the Conventional Arena Table and to METIS-X (Percent, With Verdict), and Whether METIS-X Agrees With the Reference Model.

Trace	Declarations	Maximum depth	STRATA vs. arena table	STRATA vs. METIS-X	METIS-X result
`big_scope_exit`	200 100	1	−39.0 (faster)	−53.6 (faster)	agrees
`deep_nesting`	89 949	200	−12.1 (faster)	−32.1 (faster)	agrees
`deep_shadow_400`	400	400	−51.3 (faster)	−96.4 (faster)	149 refused
`dense_50k`	50 000	0	+0.7 (n.e.)	−26.8 (faster)	agrees
`long_ids`	89 899	10	+6.9 (n.e.)	−38.9 (faster)	agrees
`lookup_heavy`	15 097	10	+12.5 (slower)	−0.1 (n.e.)	agrees
`scope_churn`	150 050	1	−24.7 (faster)	−26.3 (faster)	agrees
`shadow_heavy`	134 903	40	−21.2 (faster)	−54.3 (faster)	agrees
`short_ids`	89 646	10	−16.3 (faster)	−36.0 (faster)	agrees

TABLE 12. Declaring 32 768 Names of 150 Characters in Four Configurations: Time in Milliseconds, Their Ratio, Refused Names and Longest Probe Sequence for the Colliding Input.

Configuration	Ordinary	Colliding	Ratio	Refused	Longest
Default	6.52	1953.86	299.7	0	32 768
Keyed hash	9.33	10.80	1.2	0	106
Limit 64	3.11	7.72	2.5	32 704	64
Limit 64, keyed	5.22	5.56	1.1	0	26

TABLE 13. Exact Law Against the Real Probe-Limited Table: Observed Frequency of a Refused Push in 400 000 Trials, the Exact Value, the z-Score, Clean Trials and Lock-Step Mismatches (m Index Slots, n Keys Stored, L Probe Limit).

m	n	L	observed	exact	z	clean trials	mismatches
256	128	8	0.03311	0.03315	-0.13	254 871	0
256	160	16	0.02643	0.02667	-0.94	307 410	0
256	190	24	0.05740	0.05730	+0.26	247 303	0
256	190	40	0.01372	0.01375	-0.19	369 939	0
1024	760	48	0.01251	0.01286	-1.96	307 286	0
1024	760	96	0.00060	0.00058	+0.52	397 066	0
1024	512	24	0.00040	0.00046	-1.63	395 607	0

TABLE 14. Bounded Mode on 15 Compiler Traces (719 833 Declarations): Declarations Refused by the Probe Limit and Number of Traces Affected, for the Default and the Keyed Hash.

	Default hash		Keyed hash	
Limit	Refused	Traces	Refused	Traces
4	15 335	15	15 589	15
8	3 543	15	3 507	15
16	561	12	571	11
32	51	5	51	4
64	0	0	0	0
128	0	0	0	0
256	0	0	0	0
512	0	0	0	0

TABLE 15. Translation Units (of 169) That Fit a Fixed Budget: in Both Single-Buffer Tables, Only in STRATA, or in Neither.

Budget (KiB)	Both	Only STRATA	Neither
8	1	1	167
16	28	23	118
32	100	7	62
64	146	1	22
128	167	0	2

J. INSIDE A COMPILER

To see what the differences amount to in a whole compiler, we replaced the per-scope hash maps that chibicc [41] uses for variable scopes by STRATA and, separately, by the conventional arena table, and compiled seven of the projects with all three builds. The unmodified compiler accepts 158 of their 170 translation units; on those, the assembly produced with STRATA is byte-for-byte identical to the original on 158 units and with the arena table on 158. Over 21 interleaved rounds (Table 17), whole-project compile time with STRATA changes by between −3.7 and +2.2 percent, and with the arena table by between −3.1 and +4.3 percent; peak resident memory per unit changes by a median of 0.18 percent. In a hosted compiler neither single-buffer table matters: symbol-table work is a small part of the compile. The experiment establishes that the tables are correct in a real resolution path; it does not show a benefit for compilers that are not memory-constrained.

TABLE 16. Sweep Over the Shadowing Factor σ and the Name Length: Saving of STRATA in Smallest Fixed Buffer (Percent) and Replay Time Relative to the Conventional Arena Table (Percent).

	6 characters		12 characters		24 characters	
σ	Save	Time	Save	Time	Save	Time
1.0	0	−30.7 (faster)	0	+0.6 (n.e.)	9	+6.1 (n.e.)
1.5	24	−36.7 (faster)	25	+5.5 (n.e.)	33	+5.8 (slower)
2.0	29	−35.2 (faster)	31	+8.1 (slower)	41	+12.2 (slower)
4.0	43	−47.5 (faster)	47	−4.4 (n.e.)	57	+16.0 (slower)
8.0	50	−39.9 (faster)	55	+2.8 (n.e.)	65	+20.9 (slower)

K. AN EMULATED MICROCONTROLLER

We built a bare-metal image for a Cortex-M3 and ran it under QEMU with 64 KiB of memory, with allocation functions stubbed to fail. Five of the eight traces fit that memory; all 5 replay with a checksum over every result identical to the host's, using 492 bytes of stack, and the table's code occupies 2 715 bytes. The same 5 traces pass in bounded mode with the keyed hash and a probe limit of 512, with no refusal. At equal buffers, STRATA fits FreeRTOS in 48 KiB, Lua in 40 KiB and cJSON in 32 KiB, where the conventional table refuses 420, 2 214 and 822 operations. The emulator provides no timing, and no physical device was used.

X. AN INTERACTIVE TOOL

To make the design inspectable, the artifact includes a web interface (React [44] and Vite [45]) whose local server runs a C++ engine linked with the unmodified header. Every request

TABLE 17. End to End in chibicc: Whole-Project Compile Time With Variable Scopes on STRATA or on the Conventional Arena Table, Relative to the Original Compiler (Percent, 95 Percent Interval; Negative Is Faster).

Project	Units compiled	Original compiler (s)	STRATA		Arena table	
cJSON	2	0.11	−0.7 [−5.9, +8.8]	n.e.	+4.3 [−2.6, +10.7]	n.e.
littlefs	1	0.02	−3.7 [−5.8, +2.6]	n.e.	−3.1 [−6.4, −0.7]	faster
zlib	12	0.30	+2.2 [−3.0, +5.7]	n.e.	+0.5 [−3.9, +4.9]	n.e.
Lua	34	1.09	−0.7 [−3.5, +0.5]	n.e.	−1.8 [−2.7, −0.3]	faster
FreeRTOS	7	0.65	+1.8 [−4.1, +5.5]	n.e.	−0.5 [−7.0, +7.8]	n.e.
Mbed TLS	101	5.58	+1.4 [+0.2, +2.4]	slower	+1.0 [−0.7, +1.5]	n.e.
SQLite	1	1.00	+0.0 [−3.3, +2.8]	n.e.	−2.1 [−3.6, −0.2]	faster

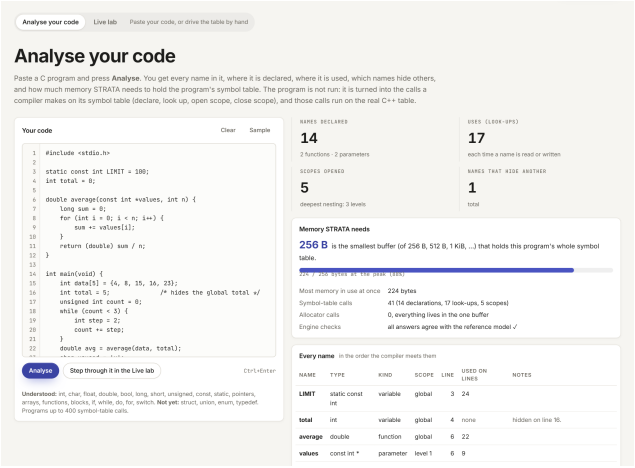


FIGURE 13. The "Analyse your code" page. A pasted C program is translated into the calls a compiler makes on its symbol table, which run on the real STRATA table in a local engine; the page lists every name with its type, scope, line and uses, the names it hides, unused and undeclared names, and the smallest buffer that holds the program.

replays the whole list of operations on a fresh table, so the C++ table is always the source of truth; after each request the engine checks the table's invariants, that every decoded record equals what `resolve` returns, and that every answer equals the reference model. The *Analyse your code* page (Fig. 13) accepts a C program, translates it into declare, lookup, enter and exit calls, and reports every name, its scope and uses, shadowing, and the smallest buffer from 256 B to 64 KiB that refuses nothing. A *Live lab* shows the buffer decoded word by word after each call and the digest before and after a refused call, and the results and theory pages draw every figure from the stored measurements and recompute the refusal law in the browser. The interface changes neither the library nor any result; its 37 automated checks compare it with the engine and with the Python programs of the analysis.

XI. DISCUSSION

What the compact design buys. Against a conventional chained table that obeys the same storage discipline, STRATA offers a buffer about one sixth smaller at a speed that is indistinguishable on most traces and up to 10.7 percent lower on the others. The ledger shows that this saving is a stored hash word and name packing; a conventional table could obtain most of it by dropping its stored hash. For a compiler on a workstation neither table matters. For a front end with tens of kilobytes, one sixth of the symbol table is memory that can go elsewhere, and on the emulated microcontroller it decided whether three workloads fit.

What the storage discipline buys. One buffer and write-after-reserve give a refusal that is simple to state and simple to test exhaustively at small scale. We consider this the more transferable result; it applies equally to the conventional table.

Why last-in-first-out removal matters beyond speed. The removal rule was adopted to avoid tombstones. Its more useful consequence is Theorem 1: because the table is always an insertion-only table, the classical exact analysis of linear probing applies to it at every moment, and Theorem 2 shows that no other deletion order would allow this for every table.

Why measurement alone was not enough. Three times in this work a measurement supported a conclusion that analysis overturned: the time penalty of Section IX-E, the probe limit of 64 that the traces suggested, and the reading of the buffer saving as a benefit of interning. We regard these as arguments for pairing replay experiments with a baseline that shares the design's constraints and with an exact analysis of the quantity being tuned.

Where the design is the wrong choice. If memory is not constrained and the input is trusted, this study gives no reason to prefer a single-buffer table. If the input is untrusted, the keyed hash costs 20.2 to 32.2 percent of replay time, and its guarantee requires a fresh key per table.

XII. LIMITATIONS AND THREATS TO VALIDITY

Mostly replay, not compilation. The traces record what a compiler's own tables were asked to do; replaying them measures a table, not a compiler built on it. The one experiment inside a compiler replaces variable scopes only, on a hosted machine, and shows no end-to-end benefit.

One machine, no hardware. All timing comes from one laptop under a hypervisor, and the microcontroller is emulated. The verdicts on individual traces are not stable across runs: in an earlier run of the same code STRATA was faster than the conventional table on two traces and slower on three. What the runs support is that the two tables are within about ten percent of each other, not a ranking trace by trace.

Workloads. The traces are C only, from two small compilers, and depend on the host's system headers. Names outside the identifier alphabet, as in mangled C++ names, take the unpacked path, which the traces exercise little.

Baselines. The conventional arena table and four further baselines were written by the authors of the table under study; they are not independent implementations. LLVM's and Clang's own containers were not measured.

The analysis. The proofs are pen-and-paper, checked by exhaustive enumeration and by comparison with the implementation, not by a proof assistant. The refusal law assumes an ideal hash; against an adversary it assumes that HalfSipHash with one and three rounds and a 64-bit key behaves as one and that keys are fresh. The key-lifetime experiment uses an idealized oracle; no timing channel was measured. A limit law for refusals on wide and deep scope trees is open.

Design limits. Names of at most 255 bytes, 16 383 nested scopes, a buffer of at most 64 MiB, one name space per table, single-threaded use.

Literature. The search for related work was limited, and several sources were consulted at the level of abstracts or documentation.

XIII. CONCLUSION

A scoped symbol table can be confined to one fixed buffer, refuse cleanly when the buffer is full, and bound the work of each call on hostile input, at a cost that we have measured and with a refusal probability that we can compute. On events logged by a C compiler for eight projects, the compact design studied here needs a buffer about one sixth smaller than a conventional single-buffer table, a saving that an exact ledger attributes to a stored hash word and name packing, and is within about ten percent of it in time, with no established difference on most traces; both are well ahead of a multi-allocation predecessor, and that larger gain belongs to the storage discipline. Because declarations are removed in the reverse order of their creation, the table is always an insertion-only table, which is the only deletion order with this property for every table and which makes the refusal probability of a probe limit exactly computable: at load three quarters, a one-in-a-million refusal needs 258 probes, not the 455 a tail bound suggests, and the law agrees with the implementation. Identifiers that collide for every seed of the default hash slow declarations 300-fold, and a keyed hash with a fresh key per table removes the effect.

The most useful next steps are a compiler that runs on the table on a physical microcontroller, workloads from languages other than C, a limit law for refusals on deep scope trees, and machine-checked proofs.

DATA AND CODE AVAILABILITY

The implementation, the tests, the compiler patches, the traces, the raw measurements of every run cited here, the verification programs of the analysis, the interactive tool, and the scripts that regenerate every number, table and figure of this article are public [38].

ACKNOWLEDGMENT

The authors thank Prof. Bhuvaneshwari M., School of Computer Science and Engineering, Vellore Institute of Technology, for guiding the project from which this work grew.

In accordance with the journal's policy on the disclosure of generated content: the implementation, the experiments, the verification programs, the derivations and the text of this article were produced with the assistance of artificial-intelligence coding assistants (Claude, Anthropic) working at the direction of the authors, as recorded in the public repository (`docs/AI_USE_DISCLOSURE.md`). Every number is produced by programs that can be re-run; the proofs are pen-and-paper and were checked by exhaustive enumeration and by statistical comparison with the implementation, not by a proof assistant. The authors take responsibility for the entire content.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[2] S. L. Graham, W. N. Joy, and O. Roubine, "Hashed symbol tables for languages with explicit scope control," in *Proc. SIGPLAN Symposium on Compiler Construction*, 1979, pp. 50–57.

[3] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge, U.K.: Cambridge University Press, 1998.

[4] S. A. Crosby and D. S. Wallach, "Denial of service via algorithmic complexity attacks," in *Proc. 12th USENIX Security Symposium*, 2003, [Online]. Available: https://www.usenix.org/conference/12th-usenix-security-symposium/denial-service-algorithmic-complexity-attacks.

[5] J.-P. Aumasson and D. J. Bernstein, "SipHash: A fast short-input PRF," in *Progress in Cryptology – INDOCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7668. Springer, 2012, pp. 489–508.

[6] LLVM Project, "ScopedHashTable.h, LLVM ADT library," [Online]. Available: https://llvm.org/doxygen/ScopedHashTable_8h_source.html. Accessed: Oct. 13, 2026.

[7] F. Bellard and TinyCC contributors, "Tiny C Compiler, commit 43c7708," [Online]. Available: https://repo.or.cz/tinycc.git. Accessed: Oct. 11, 2026.

[8] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.

[9] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, "Linear probing revisited: Tombstones mark the demise of primary clustering," in *Proc. IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, 2021, pp. 1171–1182.

[10] M. Farach-Colton, A. Krapivin, and W. Kuszmaul, "Optimal bounds for open addressing without reordering," in *Proc. 65th IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024, pp. 594–605, statements used here were read in the arXiv version 2501.02305.

[11] C. W. Fraser and D. R. Hanson, *A Retargetable C Compiler: Design and Implementation*. Redwood City, CA: Addison-Wesley, 1995, iSBN 0-8053-1670-1.

[12] D. R. Hanson, "Fast allocation and deallocation of memory based on object lifetimes," *Software: Practice and Experience*, vol. 20, no. 1, pp. 5–12, 1990.

[13] MicroPython Project, "MicroPython string interning," [Online]. Available: https://docs.micropython.org/en/latest/develop/qstr.html. Accessed: Oct. 13, 2026.

[14] R. M. Jiménez and C. Martínez, "On deletions in open addressing hashing," in *Proc. 15th Workshop on Analytic Algorithmics and Combinatorics (ANALCO)*, 2018, pp. 23–31.

[15] P. Sanders, "Hashing with linear probing and referential integrity," arXiv:1808.04602, 2018, full text read; it contains no statement of stack-ordered deletion.

[16] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in *Proc. 26th Annual Symposium on Foundations of Computer Science (SFCS)*, 1985, pp. 281–288.

[17] A. Jain and S. Singh, "METIS: Symbol-table research repository, commit 7b3dacf," 2026, [Online]. Available: https://github.com/adhyan-jain/Metis. Accessed: Oct. 11, 2026.

[18] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[19] M. Herlihy, N. Shavit, and M. Tzafrir, "Hopscotch hashing," in *Distributed Computing*, ser. Lecture Notes in Computer Science. Springer, 2008, pp. 350–364.
[20] M. Naor and V. Teague, "Anti-persistence: History independent data structures," in *Proc. 33rd Annual ACM Symposium on Theory of Computing (STOC)*, 2001, pp. 492–501.
[21] G. E. Blelloch and D. Golovin, "Strongly history-independent hashing with applications," in *Proc. 48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2007, pp. 272–282.
[22] P. Flajolet, P. Poblete, and A. Viola, "On the analysis of linear probing hashing," *Algorithmica*, vol. 22, no. 4, pp. 490–515, 1998.
[23] S. Janson, "Asymptotic distribution for the cost of linear probing hashing," *Random Structures & Algorithms*, vol. 19, no. 3–4, pp. 438–471, 2001.
[24] S. Janson and A. Viola, "A unified approach to linear probing hashing with buckets," *Algorithmica*, vol. 75, no. 4, pp. 724–781, 2016, section 11 read in the arXiv version 1410.5967.
[25] S. Janson, "Individual displacements for linear probing hashing with different insertion policies," *ACM Transactions on Algorithms*, vol. 1, no. 2, pp. 177–213, 2005.
[26] A. Viola, "Exact distribution of individual displacements in linear probing hashing," *ACM Transactions on Algorithms*, vol. 1, no. 2, pp. 214–242, 2005.
[27] M. A. Bender, A. Conway, M. Farach-Colton, W. Kuszmaul, and G. Tagliavini, "Tiny pointers," *ACM Transactions on Algorithms*, vol. 21, no. 4, pp. 1–43, 2025, theorem numbers refer to the arXiv version 2111.12800, which was read.
[28] A. Alharbi, C. Banderier, and C. Bouillaguet, "Bounded linear probing hashing," in *37th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms (AofA 2026)*, ser. LIPIcs, vol. 381, 2026, pp. 28:1–28:17, abstract, introduction, Section 3 and conclusion read.
[29] A. Pagh, R. Pagh, and M. Ružić, "Linear probing with constant independence," in *Proc. 39th Annual ACM Symposium on Theory of Computing (STOC)*, 2007, pp. 318–327.
[30] M. Thorup, "Linear probing with 5-independent hashing," arXiv:1509.04549, 2015, [Online]. Available: https://arxiv.org/abs/1509.04549.
[31] J.-P. Aumasson, D. J. Bernstein, and M. Boßlet, "Hash-flooding DoS reloaded: Attacks and defenses," Presentation slides, 2012, [Online]. Available: https://www.aumasson.jp/siphash/siphashdos_appsec12_slides.pdf.
[32] J. A. Donenfeld, "SipHash – a short input PRF," Linux kernel documentation, [Online]. Available: https://docs.kernel.org/security/siphash.html. Accessed: Oct. 13, 2026.
[33] R. J. Lipton and J. F. Naughton, "Clocked adversaries for hashing," *Algorithmica*, vol. 9, no. 3, pp. 239–252, 1993.
[34] M. Naor and E. Yogev, "Bloom filters in adversarial environments," in *Advances in Cryptology – CRYPTO 2015*, ser. Lecture Notes in Computer Science. Springer, 2015, pp. 565–584.
[35] Rust Project Developers, "rustc-hash: A speedy, non-cryptographic hashing algorithm used by rustc," [Online]. Available: https://github.com/rust-lang/rustc-hash. Accessed: Oct. 13, 2026.
[36] M. Dwass, "The total progeny in a branching process and a related random walk," *Journal of Applied Probability*, vol. 6, no. 3, pp. 682–686, 1969.
[37] J. C. Tanner, "A derivation of the Borel distribution," *Biometrika*, vol. 48, no. 1–2, pp. 222–224, 1961.
[38] S. Singh, A. Jain, A. Venkatataramana, and Arjuman, "STRATA: Source code, traces, and measurements," 2026, [Online]. Available: https://github.com/shubhisingh1510/cdesproject.
[39] A. Appleby, "MurmurHash3, in SMHasher," [Online]. Available: https://github.com/aappleby/smhasher. Accessed: Oct. 13, 2026.
[40] J.-P. Aumasson, "SipHash and HalfSipHash reference implementation," [Online]. Available: https://github.com/veorq/SipHash. Accessed: Oct. 13, 2026.
[41] R. Ueyama, "chibicc: A small C compiler, commit 90d1f7f," [Online]. Available: https://github.com/rui314/chibicc. Accessed: Oct. 11, 2026.
[42] G. Popovitch, "The parallel hashmap, version 1.4.1," [Online]. Available: https://github.com/greg7mdp/parallel-hashmap. Accessed: Oct. 13, 2026.
[43] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, 2005, pp. 41–46.
[44] Meta Open Source, "React," [Online]. Available: https://react.dev. Accessed: Oct. 20, 2026.
[45] Vite contributors, "Vite: Next generation frontend tooling," [Online]. Available: https://vite.dev/guide/. Accessed: Oct. 20, 2026.

SHUBHI SINGH is an undergraduate student with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. Research interests include compilers, data structures for constrained systems, and systems software.

ADHYAN JAIN is an undergraduate student with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. Research interests include compilers, hashing and the analysis of algorithms.

AKSHITH VENKATATARAMANA is an undergraduate student with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.

ARJUMAN is an undergraduate student with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.